WHO THE FUCK KNOWS
GAME IS THE GAME

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Student id: 11372282

Department of Computer Science

Contents

Word Count: 3289

List of Tables

List of Figures

Abstract

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Acknowledgements

I would like to thank Meuseir Bob Marley...

Todo list

List of Abbreviations

ANN Artificial Neural Network. 1, 18

ATan ArcTangent. 1, 19

CNN Convolutional Neural Network. 1, 23–26

ETraM Event-based Traffic Monitoring Dataset. 1

IoU Intersection over Union. 1, 26

LIF Leaky Integrate-and-Fire. 1, 18

ReLU Rectified Linear Unit. 1, 18

ResNet Residual Neural Network. 1, 20

SE Squeeze-and-Excitation. 1, 21, 26

SEW Spike-Element-Wise. 1, 5, 20

SNN Spiking Neural Network. 1, 17, 18, 20, 21, 23–26

Chapter 1

Technical Quality, Methodology & Evaluation

1.1 Requirements Analysis

The primary objective of this project was the development and rigorous evaluation of a high-performance Spiking Neural Network (SNN) for multi-object detection using high-definition event-camera data (the ETraM dataset). To effectively quantify the accuracy versus energy efficiency tradeoff between bio-plausible spiking architectures and traditional Convolutional Neural Networks (CNNs), the system had to be designed against a stringent set of functional and non-functional requirements. These requirements guided the iterative evolution of the neural architectures and the underlying computational pipeline.

1.1.1 Functional Requirements

To ensure a fair empirical comparison and to solve the complexities of multi-object detection on neuromorphic data, the following functional criteria were established:

- **Multi-Object Regression Capabilities:** Unlike simplified classification tasks, the system was required to perform multi-object detection capable of identifying multiple vehicles simultaneously within a single scene. This necessitated a regression head capable of outputting multiple bounding boxes per frame (defined by spatial coordinates and confidence scores) while actively suppressing "ghost detections" and resolving spatial ambiguities.

- **Architectural Parity (The "Gold Standard" Baseline):** To strictly isolate the impact of the computational paradigm (event-driven spiking versus continuous-valued activations), the baseline CNN was required to be a 1:1 non-spiking counterpart to the SNN. Both models were constrained to equivalent structures, depths and parameter counts ($\sim$ 11.7M parameters). Crucially, to eliminate data variance as a confounding factor, both the SNN and the CNN were trained on the exact same pre-processed neuromorphic dataset (the REPLICA pipeline output). This ensured that any observed discrepancies in performance or power consumption were solely attributable to the underlying network architectures, rather than differing input modalities or varied model capacity.

- **High-Definition Spatiotemporal Ingestion:** The system was required to process raw, high-definition (1280×720) event streams. Furthermore, the pipeline had to preserve the temporal dynamics of the data by segregating it into discrete time bins (e.g., $T = 10$) to allow the SNN to demonstrate temporal evidence accumulation.

- **Comprehensive Evaluation Metrics:** The evaluation suite was required to output granular metrics beyond top-line Mean Intersection over Union (IoU). This included real-time dynamic power consumption mapping, confidence calibration analysis (reliability diagrams), and quantization error tracking across varying object sizes.

1.1.2 Non-Functional Requirements & Constraints

The processing of high-definition event data at scales of up to 150 training epochs imposed severe computational bottlenecks. Consequently, the system had to satisfy critical performance and stability constraints:

- **Computational Scalability & Speed:** The SNN simulation had to be optimized for execution on local accelerator hardware (GPUs). This required the integration of Automatic Mixed Precision (AMP) to utilize Tensor Cores, memory-efficient vectorized loss functions to eliminate sequential batch loops, and the utilization of optimized C++/CUDA kernels (e.g., SpikingJelly's multi-step modes) over native Python loops.

- **Numerical Stability:** Given the reliance on 16-bit precision through AMP and

the deep accumulation of membrane potentials in the SNN, the architecture required specific safeguards against gradient explosion. This constrained the design of the detection head, mandating the use of raw logits and numerically stable loss functions (e.g., `BCEWithLogitsLoss`).

- **Hardware Extrapolability:** While constrained to local GPU execution, the benchmark outputs (e.g., dynamic power tracking) were required to be translatable into theoretical energy consumption metrics (in mJ) based on established hardware constants, facilitating projections for specialized neuromorphic chips like Intel Loihi or IBM TrueNorth.

These requirements formed the foundation for the three-generation evolution of the `SNN_MAX_REPLICA_SE` architecture detailed in Section **??**, and drove the rigorous benchmarking suite evaluated in Section **??**.

Check & Proof-read this section against rubric.

1.2 System Architecture Overview

To satisfy the stringent requirements established in Section 1.1, the overall system was designed as an end-to-end, highly modular pipeline. The modularity of this pipeline was a deliberate methodological choice to facilitate systematic design exploration. By decoupling the preprocessing, neural backbone, and regression stages, we enabled the isolation and empirical study of individual architectural modifications, ensuring that comparative findings between the spiking and continuous paradigms remained robust and controlled. The high-level data flow and structural components of the system are formalized in the block diagram presented in Figure 1.1.

As illustrated, the architecture is divided into four primary sequential stages, where the central neural backbone acts as an interchangeable module:

1. **Data Ingestion & The REPLICA Pipeline (Fixed):** Raw, asynchronous event data at a high spatial resolution of 1280×720 is ingested from the ETraM dataset. Due to the conventionally sparse and noisy nature of neuromorphic hardware outputs, the data enters the *REPLICA* preprocessing pipeline, which is heavily adapted from the methodology proposed in the original ETraM study Verma et al. 2024. While the core spatiotemporal noise filtering and bilinear temporal interpolation (to $T = 10$ bins) remain faithful to the original study,

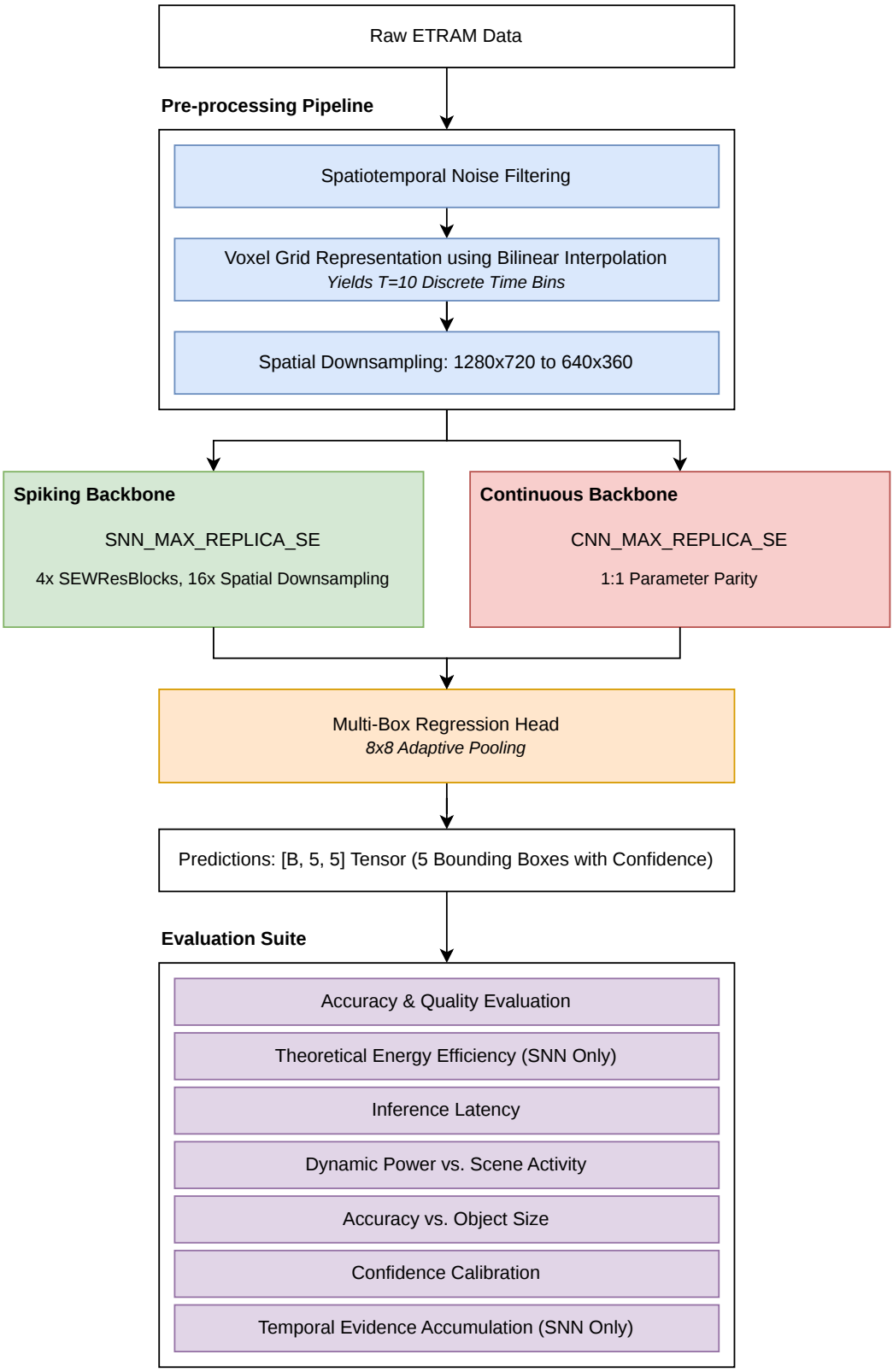

Figure 1.1: The high-level system architecture pipeline, illustrating the flow from raw ETraM high-definition event data, through the fixed REPLICA preprocessing stages, into the interchangeable neural backbone (SNN or CNN), and concluding with the multi-box regression head and the comprehensive evaluation suite.

this project included an additional spatial downsampling stage ($ds = 4 : 1280 \times 720 \rightarrow 320 \times 180$). This modification was strategically implemented to ensure training feasibility on local accelerator hardware and reduce input sparsity, thereby improving the stability of neuronal feature extraction. This stage is detailed thoroughly in Section **??**.

2. **The Neural Backbone (Interchangeable):** The preprocessed output of the *REPLICA* pipeline serves as the input to the primary feature extraction engine. This stage houses either the spiking architecture `SNN_MAX_REPLICA_SE` or its exact continuous counterpart, `CNN_MAX_REPLICA_SE`. Both backbones utilize four functional blocks incorporating Squeeze-and-Excitation (SE) mechanisms and Residual connections (`SEWResBlock`) to extract deep semantic features while performing an aggressive $16\times$ total spatial downsampling. The evolution and formal design of these backbones are detailed in Section **??**.

3. **Multi-Box Regression Head (Fixed):** The final abstracted feature maps from either backbone are fed into a unified detection head. To resolve the spatial ambiguities and "ghost detections" observed in earlier architectural iterations, the head utilizes an 8×8 Adaptive Pooling mechanism to aggregate features from 64 spatial receptive blocks. This abstracted information is then mapped to a final output tensor of shape $[B, 5, 5]$, representing the top 5 predicted bounding boxes per image. Each detection is defined by the feature set $\mathcal{F} = \{x, y, w, h, c\}$, where (x, y, w, h) denote the normalized bounding box coordinates and c represents the object confidence logit. The regression output is evaluated via a fully vectorized Multi-Box Loss function during training.

4. **Evaluation & Benchmarking Suite (Fixed):** The predictions and internal neuronal activity from the models are passed into a specialized benchmarking suite. This stage is responsible for the rigorous empirical comparison of the two architectures across five distinct dimensions: real-time dynamic power consumption, accuracy variations relative to object size (addressing the inherent "quantization error" in SNNs Cordone, Miramond, and Thierion 2022), model confidence calibration, temporal evidence accumulation, and latency observed on local hardware. These metrics were specifically selected to satisfy the project's success criteria and provide a comprehensive evaluation of the neuromorphic paradigm. This standardized evaluation framework ensures that the findings are scientifically robust and reproducible. This benchmarking suite is detailed thoroughly in

Section **??**.

Re-write this section, and link it to the Success Criteria from Introduction.

Using the modular pipeline guarantees that the SNN and CNN variants are evaluated identically, receiving the exact same preprocessed event volumes and outputting predictions through an identical regression and benchmarking framework.

1.3 Data Preprocessing: The REPLICA Pipeline

Raw neuromorphic event data is fundamentally different from standard frame-based video: instead of capturing absolute intensity at a fixed frame rate, an event camera asynchronously records per-pixel changes in log-intensity, outputting a continuous stream of events in the form $e = (x, y, t, p)$, where (x, y) are the spatial coordinates, t is the microsecond timestep, and $p \in -1, 1$ is the polarity of the event (indicating an increase or decrease in brightness).

While this asynchronous nature offers high temporal resolution and high dynamic range, raw event streams are inherently sparse, noisy and incompatible with conventional deep learning frameworks that expect dense, structured tensors. To bridge this gap, the raw Event-based Traffic Monitoring Dataset (ETraM) data (1280×720 spatial resolution) was passed through a preprocessing pipeline before network ingestion.

1.3.1 Raw Event Ingestion & Windowing

The `REPLICA` pipeline begins by reading the raw h5 event files (x, y, t, p) and the partnering npy files containing the bounding box annotations from the ETraM dataset. Once both files have been parsed, the first step taken is to apply a $100ms$ sliding window across the entire duration of the event file. "Windowing" is the process of slicing the continous event stream into chunks that represent a single instance of time for the model, thereby anchoring the asynchronous event stream into discrete, structured training instances.

To maximize training efficiency and ensure a high signal-to-noise ratio in the training distribution, the `REPLICA` pipeline adopts a Box-Central sampling strategy rather than more conventional uniform temporal partitioning, where training windows were anchored deterministically around the centroids of existing ground truth annotations. This approach ensures that 100% of the training voxels contain informative structural targets, mitigating the prevalence of 'empty-scene' noise and providing the SNN with

consistent temporal evidence to drive membrane potential accumulation prior to the target timestamp.

In this project, the time window $\Delta T = 100$ms was chosen to align with the temporal persistency of the human visual system of roughly 100ms Deodato and Melcher 2024; just as the human eye integrates luminance changes over approximately one frame of perception, the Spiking Neural Network (SNN) also integrates the asynchronous event stream over the same duration before producing a detection output.

The practical consequence of this choice is two-fold. First, a 100ms window is long enough to guarantee a minimum event density sufficient for the Leaky Integrate-and-Fire (LIF) neurons in the stem layer to accumulate membrane potential and produce meaningful spike trains across all $T = 10$ temporal bins. Second, specific to the context of vehicle detection, it is short enough that a vehicle travelling at highway speed (e.g., 120 km/h ≈ 33 ms^{-1}) displaces by only 3.3m within the window, ensuring that the bounding box annotation assigned to the window remains spatially valid and does not introduce label noise through excessive object displacement.

In combination with the Box-Central sampling approach, the pipeline applies a density filter prior to voxelization to enforce a minimum quality threshold in valid samples. For each 100ms window, the total number of raw (x, y, t, p) events is counted: windows containing fewer than `min_events` $= 500$ events are discarded, while those meeting or exceeding this threshold proceed to voxelization as valid training samples. This filter is a structural necessity for SNN training stability:

- **Prevention of dead neurons:** SNNs are metabolic models: they require energy (spikes) to learn. Training on near-empty windows rewards the optimiser for suppressing all firing, driving synaptic weights so low that even high-density frames containing real vehicles fail to trigger a spike response Eshraghian et al. 2023.

- **Firing Rate Regularization (FRR) stability:** The FRR loss penalises deviation from a target firing rate of 20% Deng et al. 2023; Yan et al. 2022. Presenting near-empty inputs forces the regularisation term to dominate the total loss, as it attempts to generate spikes in the absence of any meaningful input signal.

- **Gradient quality:** A forward pass producing no spikes yields a zero surrogate gradient Emre O Neftci, Mostafa, and Zenke 2019a. Training on empty windows is therefore mathematically degenerate since no weight update occurs, so

High Performance Computing (HPC) cycles are consumed without any learning signal.

- **Background activity suppression:** Event cameras produce a baseline level of Background Activity (BA) noise from transistor mismatch and thermal effects Gallego et al. 2022. A window containing less than 500 events is statistically likely to consist almost entirely of noise; the `min_events` threshold therefore functions as a signal intensity floor, directing the model's attention exclusively towards windows in which a meaningful scene change has occurred.

1.3.2 Signal Conditioning

THe first step of the `REPLICA` pipeline is to apply a series of signal conditioning operations to the raw event data to improve the Signal-to-Noise Ratio (SNR), which yields several benefits:

- **Prevention of LIF neuron saturation:** Without input conditioning, high-magnitude or noisy voxel values can drive every neuron in the initial stem layer to fire continuously and flood the network with uninformative spikes. By normalising the voxel tensor to the range $[0, 1]$, the input magnitude is constrained such that only neurons encoding genuine structural features,such as vehicle edges and motion boundaries, exceed their firing threshold, preserving the sparsity and selectivity of the deeper backbone stages Eshraghian et al. 2023.
- **Improved objectness discrimination:** Background artifacts inherent to High Definition (HD) event cameras, such as flickering illumination and sensor noise, generate erraneous event clusters that appear similar to small objects. By rejecting these low-activity events during the Background Activity Filtering (BAF) denoising step, the network is not required to expend representational capacity on learning to suppress noise, and can instead focus on learning the geometric structure of real vehicles Eshraghian et al. 2023; Roy, Jaiswal, and Panda 2019a.
- **Reduced synaptic operations and neuromorphic energy efficiency:** Since SNNs consume energy only upon spike emission, every erraneous background event that is suppressed during preprocessing directly reduces the number of Synaptic Operations (SOPs) at inference time. This establishes a direct link between preprocessing quality and the power efficiency of the architecture when deployed on neuromorphic hardware Roy, Jaiswal, and Panda 2019a.

The Signal Conditioning segment of the `REPLICA` pipeline was lifted directly from the original ETraM study, since it was designed specifically to suit the characteristics of the dataset Verma et al. 2024.

1.3.3 Spatiotemporal Voxelization

To make the asynchronous event data compatible with neural network layers, a Voxelization process is performed to transform the continuous stream into a structured tensor. This process consists of three core operations:

- **Temporal Binning:** The total duration of an event sample (e.g., a 100ms window) is divided into 10 discrete time bins ($T = 10$).
- **Polarity Splitting:** Events are separated into two distinct channels based on their polarity ($C = 2$), representing positive and negative brightness changes.
- **Event Accumulation (Histogramming):** Within each discrete temporal bin and polarity channel, the individual asynchronous events are aggregated at their respective spatial coordinates (x, y). Specifically, the value of each "pixel" in the resulting spatial grid becomes the total count of events (of that specific polarity) that occurred at that exact location during that fraction of time. This process effectively converts the sparse, continuous point-cloud of events into a sequence of dense 2D frames—or event histograms—which standard convolutional and spiking layers can natively process.

This accumulation process preserves the temporal distribution of the events while generating a dense 4D spatiotemporal tensor of shape $[10, 2, 720, 1280]$ (representing Time $\times$ Channels $\times$ Height $\times$ Width).

1.3.4 Downsampling & Normalization

Following spatiotemporal voxelization, the voxelized data undergoes a dual-phase refinement consisting of spatial downsampling and amplitude normalization. This is the final polishing step of the pipeline, transforming the high-resolution event counts into a stabilized, low-memory format that the neural backbones can process efficiently.

Spatial Downsampling (16-fold Data Reduction)

The generated high-definition voxel grid (1280×720) is subjected to a 4×4 Max-Pooling operation, reducing the spatial resolution to 320×180. This 16-fold reduction in data volume is critical for managing the high-dimensional state-space of the SNN backbone. Because SNNs must store neuron membrane potentials across time, an HD model would require massive amounts of GPU memory. By aggressively reducing the spatial dimensions, we ensure the backbone scalability, making it computationally feasible to train the deep 4-block architecture over extended epochs.

Furthermore, utilizing Max-Pooling (as opposed to Average-Pooling) acts as a feature preserver: it retains the strongest event signal within each 4×4 area, helping the network maintain the crisp, high-contrast edges of the vehicles even at a lower resolution.

It should be noted that the downsampling factor $ds = 4$ was heavily influenced by the HPC cluster's GPU memory constraints; without using $ds = 4$, the data would not fit into the GPU memory, or would require a batch size of 1, making training infeasible within the time constraints of this project.

Max-Normalization (Input Calibration)

Subsequently, max-normalization is performed to scale the raw event accumulation counts into a continuous $[0, 1]$ interval, by dividing every pixel in the temporal volume by the single highest event count found within that 10-bin voxel.

This normalization serves as a fundamental safeguard against neural saturation. LIF nodes have fixed firing thresholds (in this architecture, $V_{th} = 0.2$) - if a raw, unscaled event count of 50 were fed into the network, the first-layer neurons would fire instantaneously and repeatedly, effectively "screaming" and drowning out all other spatiotemporal information. By enforcing input calibration, we ensure that the input magnitudes are calibrated relative to the LIF thresholds, facilitating stable surrogate gradient propagation during the 150-epoch training cycle.

1.3.5 Multi-Object Label Alignment

The final stage of the pipeline implements a Multi-Object Label Alignment strategy, which is fundamental to the architecture's multi-object detection capabilities. This stage transforms individual, asynchronous bounding box annotations into structured, scale-independent target tensors suitable for the network's multi-box regression head.

Timestamp-Based Grouping (Unified Scene Representation)

Instead of creating isolated training samples for every individual vehicle, the pipeline parses the ground truth annotations and groups all vehicles that exist at the exact same microsecond (t). This process merges individual detections into a Unified Scene Representation, which is crucial because it eliminates "label noise"; presenting the network with identical visual input but different single-car targets across multiple samples would force the model to average its predictions, resulting in blurry, low-confidence bounding boxes with poor Intersection over Union (IoU) scores.

Fixed-Capacity Padding

Deep learning models inherently require consistent tensor dimensions for batched training. To accommodate scenes with varying numbers of vehicles, we enforce a Fixed-Capacity Interface of exactly 5 bounding boxes per frame. If a scene contains 2 vehicles, the remaining 3 slots are populated with "empty" zero-padded boxes. If a frame contains no vehicles, 5 empty boxes are provided. This ensures every target label is a consistent $[5,5]$ tensor. The limit of 5 boxes was empirically selected based on an analysis of the ETraM validation set, where the vast majority of frames contain 3 or fewer vehicles, making 5 a safe upper bound for this specific traffic monitoring environment.

Binary Masking

To enable the network to distinguish between genuine vehicle targets and zero-padded instances, the tensor utilizes a Binary Masking dimension. For every box, the target tensor stores five values: $[Center_X, Center_Y, Width, Height, Mask]$.

- For a real object, the mask value is set to 1.0.
- For a padded or empty box, the mask value is set to 0.0.

This mask acts as the objectness confidence target. It is utilized by the Multi-Box Loss function to selectively apply coordinate gradients only to valid targets, while simultaneously regularizing the model's confidence head to output 0.0 for empty regions.

Normalization to Sensor Space

Finally, all spatial parameters (center coordinates, width, and height) are originally provided in raw pixel values (e.g., $x = 842$). These are normalized by dividing by the raw

sensor width (1280) and height (720), scaling all coordinates into the continuous range $[0.0, 1.0]$. This provides a Scale-Independent Signal, ensuring the regression head can learn the underlying geometry robustly, regardless of the spatial downsampling factors applied earlier in the pipeline.

1.3.6 Chunking for High-Throughput I/O

To optimize throughput on the HPC cluster, the precomputed dataset was encapsulated into a High-Throughput Chunking architecture. Without this optimization, a 150-epoch training run on the high-definition event data would likely take days due to severe I/O bottlenecks.

The Chunking Mechanism

Rather than saving 10,000 individual files representing every single frame, the pipeline bundles the data into large, contiguous blocks, or "chunks" (e.g., 250 or 500 samples per file), where the chunk size is defined by the memory constraints of the environment used for training. Each chunk is saved as a single, compressed PyTorch (`.pt`) file containing two massive tensors: the spatiotemporal voxels and their corresponding aligned targets. To manage this structure, a global `manifest.json` file acts as a map, recording exactly which sample index resides in which chunk and specifying its byte-offset.

Mitigating HPC Bottlenecks

This chunking strategy was an absolute necessity for efficient execution on the cluster, addressing three primary hardware constraints:

- **Solving the "Small File Problem" (Metadata Latency):** HPC systems typically utilize a Network Attached Storage (NAS/NFS) architecture. Opening a file on a network drive incurs high *Metadata Latency*. If the GPU were forced to open 10,000 independent files per epoch, it would spend the vast majority of its time idling while waiting for the network to locate the next file. By encapsulating the data into chunks, the HPC only needs to open a fraction of the files (e.g., 40 files for 10,000 samples). This allows the system to perform massive *Sequential I/O* reads, which are orders of magnitude faster than random access reads on networked drives.

- **Bypassing CPU Precomputation Bottlenecks:** In deep learning pipelines, the CPU often acts as the data loader for the GPU. Preprocessing 1280×720 event data into voxel grids on-the-fly is highly computationally expensive.; by enforcing strict *Precomputation* and chunking everything prior to training, the training script can simply copy the structured tensors from disk directly into VRAM. This ensures the CPU is no longer a bottleneck, maintaining *Maximum GPU Utilization* and enabling iteration speeds as low as 160ms/it.

- **Efficient Memory Caching (LRU):** The chunked architecture enabled the implementation of a custom `VoxelDiskDataset` featuring a Least Recently Used (LRU) memory cache. Because the data is bundled, the HPC can load entire blocks of 250 samples directly into its extensive RAM (e.g., 120GB). Consequently, during subsequent epochs, the model frequently bypasses disk I/O entirely, pulling chunks directly from RAM and making data loading nearly instantaneous.

By utilizing chunk-based sequential I/O and a memory-mapped caching layer, the *REPLICA* pipeline successfully bypassed the CPU-loading bottleneck, ensuring maximum GPU utilization and reducing the total 150-epoch training time from a projected several days to approximately six/ seven hours.

1.3.7 REPLICA Pipeline: Visualized

Figure 1.2 details the order of which the aformentioned preprocessing steps are applied to the raw event data.

1.4 Spiking Neural Network (SNN) Architecture

The core focus, effort and innovation of this project is the development and optimization of the `SNN_MAX_REPLICA_SE` architecture. This model is explicitly designed to process high-definition neuromorphic event data while adhering to the computational paradigms of SNNs. By prioritizing event-driven, binary communication, the architecture aims to drastically reduce both the dynamic power consumption and the computational complexity compared to traditional deep learning models, making it superior for power and time constrained environments.

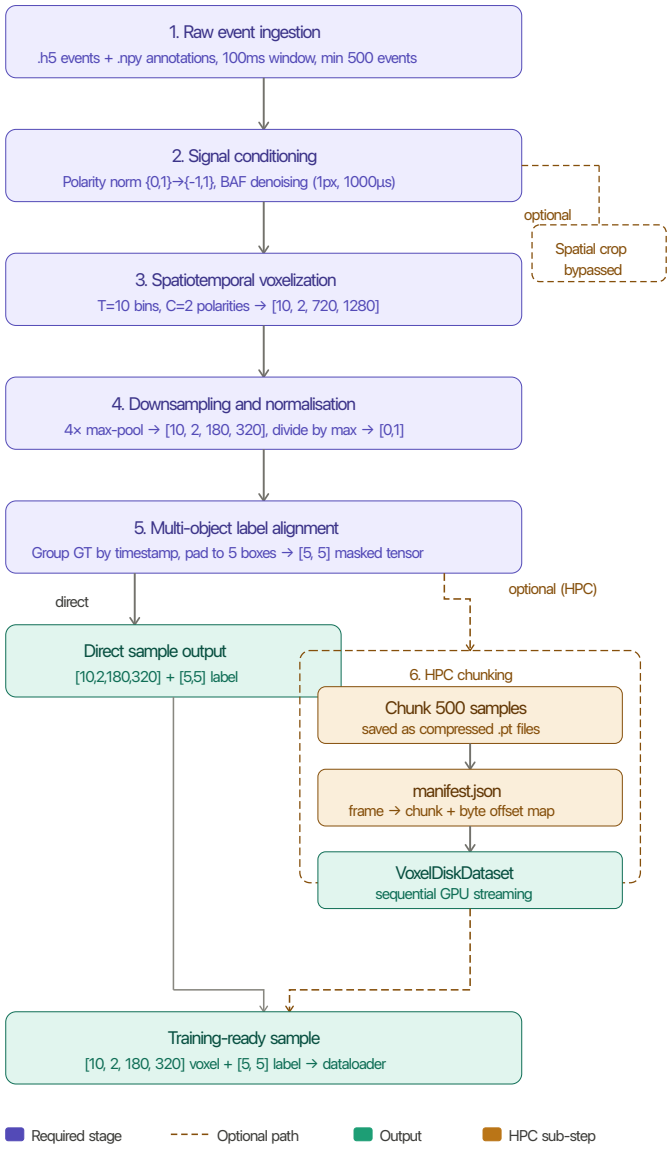

Figure 1.2: Overview of the spatiotemporal preprocessing pipeline applied to the ETraM dataset. Raw events are voxelized into a 4D tensor, spatially downsampled, and aligned with multi-object ground truth labels to produce a final sample of shape `[10, 2, 180, 320]` with a corresponding label tensor of shape `[5, 5]`.

1.4.1 Spatiotemporal Input & Multi-Step Execution

The raw ETraM event data is initially preprocessed to a spatial resolution of 640×360 with $T = 10$ discrete temporal bins. Consequently, the network ingests a massive, high-dimensional spatiotemporal tensor of shape $[B, 10, 2, 360, 640]$ (Batch, Time, Polarity, Height, Width).

Given the high spatial dimensionality and the iterative nature of spiking neurons, native Python loop execution over the time dimension proved computationally restrictive. To overcome this bottleneck, the entire architecture was refactored to utilize the SpikingJelly framework's optimized Multi-Step (`step_mode='m'`) execution paradigm Fang, Chen, et al. 2023; instead of iterating through the 10 time bins sequentially, the entire spatiotemporal tensor is passed directly into custom-compiled CUDA kernels. This low-level optimization enables high-speed, parallelized execution of the temporal dynamics, making the extensive benchmarking of this HD dataset feasible on local GPU hardware, and reducing overall training time from approximately 16 hours to just 7.

1.4.2 Neuron Dynamics: Leaky Integrate-and-Fire (LIF)

At the foundation of the architecture lies the LIF neuron model, which governs the temporal dynamics and binary spike generation of the network. Unlike traditional Artificial Neural Networks (ANNs) that utilize continuous activation functions such as ReLU, the LIF node maintains an internal state: the membrane potential $V[t]$, which updates iteratively across discrete time steps $t \in [1, T]$.

The Forward Pass (Spike Generation)

The dynamics of the LIF neuron at a given spatial position and time step t are governed by the integration of incoming pre-synaptic spatial features $X[t]$ and an inherent leak factor τ. The hidden membrane potential $H[t]$ prior to spiking is mathematically defined as:

$$H[t] = V[t-1] + \frac{1}{\tau}(X[t] - (V[t-1] - V_{reset})) \tag{1.1}$$

When $H[t]$ exceeds a predefined threshold V_{th}, the neuron emits a binary spike $S[t] \in \{0, 1\}$ to the subsequent layer, governed by the Heaviside step function $\Theta(x)$:

$$S[t] = \Theta(H[t] - V_{th}) \tag{1.2}$$

Following a spike ($S[t] = 1$), the neuron undergoes a hard reset, instantly dropping its potential $V[t]$ to V_{reset}. If no spike is emitted ($S[t] = 0$), the potential naturally decays, ensuring that subsequent layers only perform computations (Synaptic Operations) upon receiving active events Sengupta et al. 2019. This "activation sparsity" is the primary driver of the SNNs exceptional energy efficiency Roy, Jaiswal, and Panda 2019b.

The Backward Pass (Surrogate Gradients)

Training deep SNNs via backpropagation presents a fundamental mathematical challenge, since the Heaviside step function governing spike generation (Equation 2) is non-differentiable. Its derivative is zero everywhere except at the threshold V_{th}, where it is infinite. Using conventional backpropogation here would immediately result in zeroed gradients, halting the learning process - this is a phenomenon known as the "dead neuron" problem.

To resolve this, the network employs a Surrogate Gradient during the backward pass Emre O. Neftci, Mostafa, and Zenke 2019b. While the forward pass remains strictly binary, the backward pass approximates the step function using a smooth, differentiable curve. This specific architecture uses the ArcTangent (ATan) function:

$$\frac{\partial S}{\partial H} \approx \frac{\alpha}{2\left(1+\left(\frac{\pi}{2}\alpha(H-V_{th})\right)^2\right)} \tag{1.3}$$

Where α is a hyperparameter controlling the steepness of the surrogate curve. This mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution.

1.4.3 Backbone: SEW-ResNet & Attention

To extract high-level semantic features and manage the computational complexity of the high-definition input, the architecture utilizes a deep residual backbone divided into five sequential stages, as outlined in Table 1.1.

Spatial Downsampling

As illustrated, the architecture implements an aggressive $16\times$ total spatial downsampling strategy to compress the initial 640×360 input. This is achieved through strided

Stage	Name	Resolution	Channels	Primary Operation
1	Initial Stem	90×160	64	Stride-2 Conv + LIF (Downsamples HD input)
2	Backbone Alpha	22×40	256	Blocks 1 & 2 (SEWResBlocks with Stride-2)
3	Attention Bridge	22×40	256	Squeeze-and-Excitation (SE) Global Firing Recalibration
4	High-Res Backbone	22×40	512	Blocks 3 & 4 (SEWResBlocks with Stride-1)
5	Temporal Fusion	22×40	512	Summation of spikes across all T=10 timesteps

Table 1.1: Detailed breakdown of the five sequential stages in the `SNN_MAX_REPLICA_SE` backbone.

convolutions applied within the Initial Stem and Backbone Alpha, progressively reducing the spatial dimensions to a highly compressed 22×40 grid while expanding the channel width up to 512.

Spike-Element-Wise (SEW) Residual Blocks

Building deep SNNs presents significant challenges due to the "vanishing spike" problem inherent in standard residual connections. In a standard Residual Neural Network (ResNet), the residual addition occurs *before* the activation function. However, if we follow this standard and add two binary spike trains, the result is a non-binary, continuous value (e.g., $1 + 1 = 2$), which destroys the energy-efficient, event-driven nature of the network Fang, Yu, et al. 2022.

To overcome this, the `SNN_MAX_REPLICA_SE` architecture adopts the SEW residual paradigm Fang, Yu, et al. 2022, instantiated as `SEWResBlock` units throughout the backbone. Unlike standard Spiking ResNets, SEW ResBlocks apply the element-wise residual operation *after* the spiking activation function, guaranteeing that the output remains strictly binary. Fang et al. Fang, Yu, et al. 2022 prove that this reordering is both necessary and sufficient to achieve identity mapping in a purely spike-based network, resolving the degradation problem without sacrificing neuromorphic compatibility. In this architecture, this property is exploited to stack 4 SEWResBlocks across the backbone stages (Stages 2 and 4 in 1.1), enabling representational depth that would otherwise cause spike degradation under a standard residual formulation.

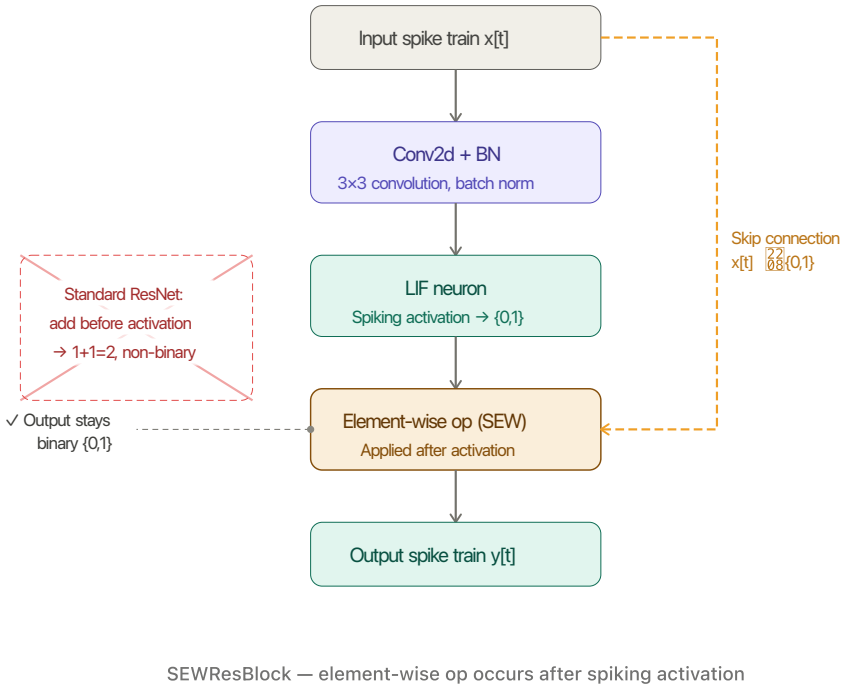

Figure 1.3: The SEW residual block (`SEWResBlock`) as proposed by Fang, Yu, et al. 2022.

Structural Attention: SE-Block

In deep learning, a bottleneck refers to a point in the neural network where the information flow is forced through its most compressed and abstract representation. In this `SNN_MAX_REPLICA_SE` architecture, the bottleneck exists at the transition point between stages 2 and 3 of the backbone, and represents two things:

1. **A Spatial Constraint (Resolution Limit):** At this stage, the original 180×320 spatiotemporal input has been downsampled by a factor of 8x via the strided convolutions in the stem and early residual block, resulting in a highly compressed22×40 feature map. This spatial bottleneck is critical because each remaining "pixel" must now represent a large 8×8 receptive field of the original HD sensor data, forcing the network to discard fine-grained geometric redundancy while retaining the core structural boundaries of multiple detected objects.

2. **A Semantic Shift (High-Level Abstraction):** This juncture marks the definitive

pivot where the network transitions from extracting low-level geometric primitives, such as spike-edge orientation and motion direction, to synthesizing complex semantic concepts. Without an explicit selection mechanism, the network at this depth is susceptible to noise and artifacts from the HD event stream, which can blur the distinction between legitimate targets and background interference.

To both mitigate and solve these bottleneck constraints, a Squeeze-and-Excitation (SE) mechanism is introduced in Stage 3 of the backbone, sitting directly at the transition point Hu et al. 2019. The SE block operates by performing a global average pooling of the 256 feature channels (the "Squeeze" phase), followed by two fully connected layers that calculate a weighting factor for each channel (the "Excitation" phase). This effectively mitigates the spatial constraint by allowing the network to incorporate global context into its local representations, compensating for the loss of resolution.

When applied in the context of an SNN, this acts as a dynamic firing modulator: the backbone's activations are re-calibrated by looking at the global context of the entire frame, magnifying the most informative channels whilst suppressing those containing artifacts. By applying this recalibration, the SE block ensures that only the most relevant "Objectness" (the probability that a specific region of an image contains a generic object of interest rather than background noise) features are passed into the final high-capacity blocks, effectively mitigating the semantic ambiguity before the final bounding box regresssion.

1.4.4 Firing Rate Regularization (FRR)

A common failure mode in deep SNN training is network silencing (zero spikes emitted) or over-saturation (firing continuously at every timestep), both of which halt gradient flow and destroy energy efficiency. To constrain the network, a custom Firing Rate Regularization (FRR) Metabolic Penalty was integrated into the training pipeline.

The FRR applies a squared error loss to the network's average firing rate:

$$L_{frr} = \lambda \cdot (Rate_{avg} - Target)^2 \tag{1.4}$$

This biologically inspired metabolic constraint Roy, Jaiswal, and Panda 2019a forces the 11.7M parameter backbone to maintain a selective but informative firing rate (e.g., $Target = 20\%$) across its hidden layers. Penalising derivation from a target firing rate is an established technique for preserving activation sparsity in directly

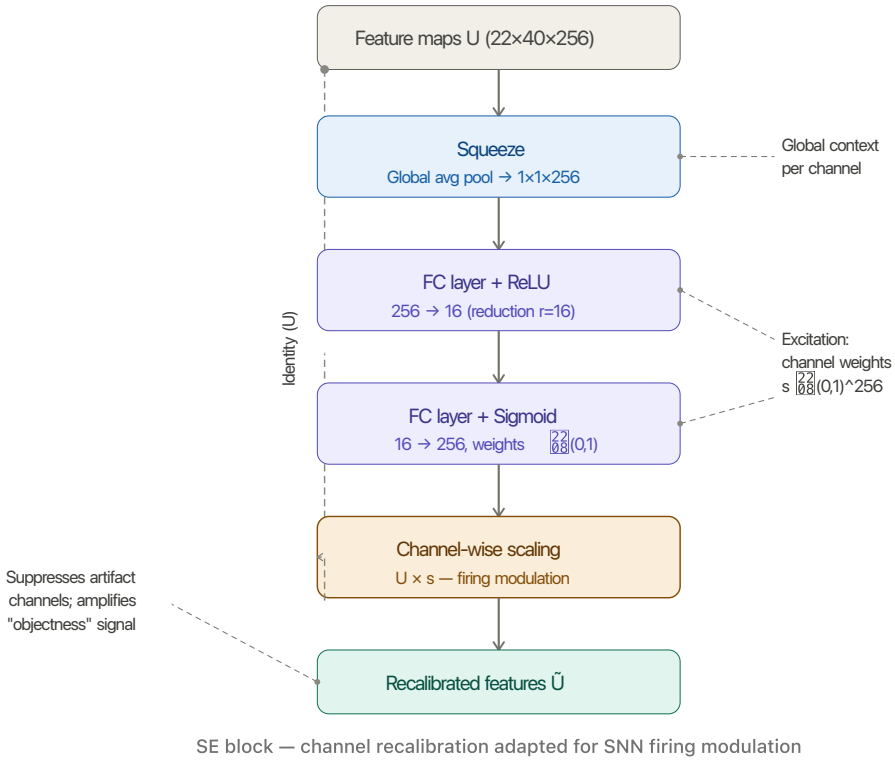

Figure 1.4: The Squeeze-and-Excitation (SE) block Hu et al. 2019 as applied in Stage 3 of the `SNN_MAX_REPLICA_SE` backbone ($22\times40\times256$).

trained SNNs Deng et al. 2023; Yan et al. 2022. Adding this penalty to the model's loss ensures high information flow while strictly preserving the activation sparsity required for efficient neuromorphic deployment.

1.4.5 Multi-Box Regression Head

Following the final backbone stage, the abstract spatiotemporal feature maps are fed into a unified detection head. To process the spatial information effectively and mitigate spatial ambiguities observed during the iterative design process (detailed in Section **??**), the head utilizes an 8×8 Adaptive Average Pooling mechanism.

This pooling operation aggregates the features into 64 distinct spatial receptive blocks. The pooled features are then mapped via final convolutional and linear layers to an output tensor of shape $[B,5,5]$, where B represents the batch size. The final two dimensions encode the top 5 predicted bounding boxes globally per image, with each box defined by a feature tuple $\mathcal{F} = \{x,y,w,h,c\}$. Here, (x,y,w,h) represent the

normalized spatial coordinates and dimensions of the bounding box, and c denotes the object confidence logit. This formulation allows the network to predict multiple vehicles simultaneously, which are subsequently evaluated using a vectorized Multi-Box Loss function.

Add model architecture diagram.

Check & Add model hyperparameter table.

1.5 Convolutional Neural Network (CNN) Baseline

The Convolutional Neural Network (CNN) architecture `CNN_MAX_REPLICA_SE` was engineered as the primary baseline to evaluate the SNN against. It is a 11.73M parameter CNN specifically designed to mirror the SNN architecture, satisfying the "Architectural Parity" functional requirement specified in 1.1.1. This ensures that the only variable being tested in this comparative study is the neural activation mechanism (event-driven binary spikes versus continuous-valued floats), making the comparison immune to common criticisms regarding unmatched or unoptimized baselines.

1.5.1 Structural Configuration & Layer-Wise Mapping

The CNN mirrors the SNN's configuration strictly 1:1, replacing every temporal and spiking component with a standard continuous equivalent. Table 1.2 outlines this direct mapping.

SNN Component	CNN Equivalent	Technical Purpose
Spatiotemporal Voxel `[B, 10, 2, H, W]`	Flattened Channel Input `[B, 20, H, W]`	Translates 10 time bins into a format the respective network can ingest.
LIF Neuron (Leaky Integrate-and-Fire)	ReLU Activation	Provides the nonlinearity; SNN uses discrete pulses, CNN uses continuous values.
Binary Spikes $\{0,1\}$	Continuous Floats $[0,\infty)$	Represents the information "currency" being passed between layers.
Surrogate Gradient	Exact Derivative (Standard)	Enables backpropagation through the non-differentiable spiking threshold.
SEWResBlock (Spiking Shortcut)	Standard Residual Block	Implements identity mapping to prevent vanishing gradients during training.
Multi-Step Mode (`step_mode='m'`)	Single spatial pass	Processes the temporal window efficiently on the GPU.
Squeeze-and-Excitation (Spike-based)	Squeeze-and-Excitation (Float-based)	Recalibrates channel importance based on global frame context.
Temporal Summation ($\sum_{t=1}^{T} S_t$)	N/A (Direct Feature Pass)	Aggregates 100ms of evidence into a single map for the regression head.
Multi-Box Head (5 Boxes)	Multi-Box Head (5 Boxes)	Predicts coordinates and objectness scores for 5 targets simultaneously.
Synaptic Operations (SOPs)	Floating Point Ops (FLOPs)	The metric used to quantify and compare hardware energy efficiency.

Table 1.2: 1:1 Architectural Mapping (SNN vs. CNN)

1.5.2 Input Strategy: Channel-Wise Temporal Integration

Unlike the SNN, which processes event data sequentially over discrete time steps T, the CNN baseline utilizes a flattened temporal input strategy. Instead of processing 10 timesteps sequentially, it concatenates the $T = 10$ bins (each containing 2 polarity channels) into a single input tensor of 20 channels, changing the input shape from $[B, 10, 2, H, W]$ to $[B, 20, H, W]$.

This approach allows the CNN to correlate events across the entire 100ms window in a single spatial pass. This channel-wise stacking of event bins is a standard and effective methodology for feeding asynchronous event-camera data into traditional, non-spiking CNN architectures Gehrig et al. 2019.

1.5.3 Backbone: Identity Mapping & Residual Connections

The CNN backbone employs deep residual learning, utilizing four Residual Blocks that expand to 512 channels in the final stage. To satisfy the functional requirement of Architectural Parity, the spiking `SEWResBlocks` from the SNN were replaced with Standard Residual Blocks. The standard structure within these blocks is Conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ BatchNorm + Shortcut $\rightarrow$ ReLU.

The inclusion of these skip connections (shortcuts) is vital for enabling the CNN network to avoid the same vanishing gradient problem seen in its SNN counterpart, and successfully learn complex vehicle geometries across the 320×180 field of view He et al. 2015.

1.5.4 SEWResBlock vs Standard ResBlock

The core mathematical difference between the two architectures lies in their summation logic and the resulting information flow across the shortcut connection:

- **Standard ResBlock (CNN):** Performs an accumulative floating-point addition ($y = f(x) + x$). The continuous intensity of the residual path is directly added to the identity path before the final ReLU activation. Here, the shortcut carries the *raw input features* forward. This mechanism allows the model to learn the "difference" (the residual) needed to improve the representation, focusing on information preservation He et al. 2015.

- **SEWResBlock (SNN):** Performs a relational element-wise spiking operation *after* the activation function. Because adding two binary spikes (e.g., $1 + 1 = 2$)

produces a non-binary value, the SNN uses a spike-merging function (such as ADD or AND) to preserve the discrete $\{0,1\}$ state. Crucially, the SNN shortcut carries the *membrane history* forward, prioritizing temporal integrity.

Table 1.3 summarizes these foundational differences in identity mapping.

Feature	**Standard ResBlock (CNN)**	**SEWResBlock (SNN)**
Summation	Accumulative ($y = f(x) + x$)	Relational (Spiking Element-Wise)
Output Type	Continuous Float $[0, \infty)$	Binary Spikes $\{0,1\}$
Primary Benefit	Prevents vanishing gradients	Prevents spike vanishing
Mathematical Goal	Information preservation	Temporal integrity

Table 1.3: Comparison of identity mapping between Standard ResBlocks and SEWResBlocks.

This distinction is particularly relevant for high-speed neuromorphic data: while the Standard ResBlock accumulates continuous values that effectively blur fast movements into a single averaged float over the 100ms window, the SNN equivalent preserves exact spike timing, maintaining the temporal precision required for the regression head.

1.5.5 Attention Mechanism: SE-Block

The CNN network encounters the same informational bottlenecks as its SNN counterpart, detailed in Section 1.4.3. Consequently, it implements the same architectural solution via an SE Block Hu et al. 2019. However, the `CNN_MAX_REPLICA_SE` implements this mechanism in the continuous domain: as opposed to utilizing spike rates, the CNN employs a float-based approach.

During the 'Squeeze' segment, the CNN utilizes global average pooling to calculate the mean spatial intensity of features per channel, rather than calculating a temporal firing rate. Subsequently, during the 'Excitation' phase, the block outputs a learned coefficient that scales the input activations. This effectively recalibrates the channel-wise feature responses, as opposed to modulating discrete membrane potentials.

While the integration of an SE Block in a standard CNN improves feature representation, it does not confer the same critical hardware-level energy sparsity benefits as it does within an SNN. The primary advantage of the SE mechanism in this specific context is the enforcement of structural sparsity in the spiking domain. Nevertheless,

to strictly adhere to the Architectural Parity requirement established in Section 1.1.1, the continuous equivalent of this mechanism was preserved in the baseline.

1.5.6 The "Gold Standard" Upper Bound

By constructing this custom CNN mirror, the baseline serves as the upper theoretical bound of this specific architecture's capability on the ETraM dataset. For example, if the CNN baseline achieves a Mean IoU of 48%, this represents the maximum performance extractable by this parameter configuration. If the corresponding SNN achieves 43% IoU, the 5% gap explicitly represents the "quantization cost" of switching from 32-bit continuous floats to 1-bit binary spikes Cordone, Miramond, and Thierion 2022. Adhering to this methodology ensures that the evaluation of the neuromorphic paradigm is both fair and scientifically sound.

Add model architecture diagram.

Check & Add model hyperparameter table.

Bibliography

Cordone, Loïc, Benoît Miramond, and Philippe Thierion (2022). *Object Detection with Spiking Neural Networks on Automotive Event Data*. arXiv: 2205.04339 [cs.CV]. URL: https://arxiv.org/abs/2205.04339.

Deng, Lei et al. (2023). "Comprehensive SNN Compression Using ADMM Optimization and Activity Regularization". In: *IEEE Transactions on Neural Networks and Learning Systems* 34.6, pp. 2791–2805. DOI: 10.1109/TNNLS.2021.3109064.

Deodato, Michele and David Melcher (Dec. 2024). "Continuous temporal integration in the human visual system". In: *Journal of Vision* 24.13, pp. 5–5. ISSN: 1534-7362. DOI: 10.1167/jov.24.13.5. eprint: https://arvojournals.org/arvo/content_public/journal/jov/938697/i1534-7362-24-13-5_1733390407.54037.pdf. URL: https://doi.org/10.1167/jov.24.13.5.

Eshraghian, Jason K et al. (2023). "Training Spiking Neural Networks Using Lessons From Deep Learning". In: *Proceedings of the IEEE* 111.9, pp. 1016–1054. DOI: 10.1109/JPROC.2023.3308088.

Fang, Wei, Yanqi Chen, et al. (2023). *SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence*. arXiv: 2310.16620 [cs.NE]. URL: https://arxiv.org/abs/2310.16620.

Fang, Wei, Zhaofei Yu, et al. (2022). *Deep Residual Learning in Spiking Neural Networks*. arXiv: 2102.04159 [cs.NE]. URL: https://arxiv.org/abs/2102.04159.

Gallego, Guillermo et al. (2022). "Event-Based Vision: A Survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1, pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413.

Gehrig, Daniel et al. (2019). *End-to-End Learning of Representations for Asynchronous Event-Based Data*. arXiv: 1904.08245 [cs.CV]. URL: https://arxiv.org/abs/1904.08245.

He, Kaiming et al. (2015). *Deep Residual Learning for Image Recognition*. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385.

Hu, Jie et al. (2019). *Squeeze-and-Excitation Networks*. arXiv: 1709.01507 [cs.CV]. URL: https://arxiv.org/abs/1709.01507.

Neftci, Emre O, Hesham Mostafa, and Friedemann Zenke (2019a). "Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks". In: *IEEE Signal Processing Magazine* 36.6, pp. 51–63. DOI: 10.1109/MSP.2019.2931595.

— (2019b). "Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks". In: *IEEE Signal Processing Magazine* 36.6, pp. 51–63. DOI: 10.1109/MSP.2019.2931595.

Roy, Kaushik, Akhilesh Jaiswal, and Priyadarshini Panda (2019a). "Towards spike-based machine intelligence with neuromorphic computing". In: *Nature* 575.7784, pp. 607–617.

— (Nov. 2019b). "Towards spike-based machine intelligence with neuromorphic computing". In: *Nature* 575.7784, pp. 607–617. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1677-2. URL: https://pubmed.ncbi.nlm.nih.gov/31776490/.

Sengupta, Abhronil et al. (2019). *Going Deeper in Spiking Neural Networks: VGG and Residual Architectures*. arXiv: 1802.02627 [cs.CV]. URL: https://arxiv.org/abs/1802.02627.

Verma, Aayush Atul et al. (June 2024). "eTraM: Event-based Traffic Monitoring Dataset". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 22637–22646.

Yan, Yulong et al. (2022). "Backpropagation With Sparsity Regularization for Spiking Neural Network Learning". In: *Frontiers in Neuroscience* 16, p. 760298. DOI: 10.3389/fnins.2022.760298.